DUNE: ARRAKIS DOMINION DISTRIBUTED

Game Design Document Completo

Versión: 1.0
Fecha: Abril 2026
Tipo: Práctica Académica - Programación en Entornos Distribuidos
Lenguaje: C# / .NET
Plataforma: Windows / Consola

Resumen Ejecutivo

El presente documento constituye el Game Design Document completo para el proyecto "Dune: Arrakis Dominion Distributed", una simulación estratégica desarrollada como práctica académica para la asignatura de Programación en Entornos Distribuidos del Grado en Ingeniería en Tecnologías para la Animación y los Videojuegos.

El jugador asume el papel de representante de una Casa Menor autorizada a operar enclaves de explotación, investigación y exhibición en el planeta Arrakis, el desierto más valioso del universo conocido. La gestión incluye la alimentación y reproducción de criaturas nativas, la construcción de instalaciones especializadas, la atracción de visitantes mediante exhibiciones públicas y la maximización de beneficios económicos.

El sistema está diseñado como una aplicación distribuida con componentes separados que cooperan mediante comunicación síncrona y asíncrona, permitiendo persistencia del estado, trazabilidad de acciones y escalabilidad futura.

Tabla de Contenidos

1. Introducción y Contexto
2. Lore y Ambientación
3. Análisis de Requisitos
4. Diseño del Sistema de Enclaves
5. Diseño del Sistema de Criaturas
6. Diseño del Sistema de Instalaciones
7. Diseño del Sistema de Visitantes
8. Diseño del Sistema de Recursos
9. Sistema de Rondas y Simulación
10. Arquitectura Distribuida
11. Diseño del Centro de Mando
12. Gestión de Errores y Robustez
13. Estructuras de Datos y Algoritmos
14. Plan de Desarrollo
15. Apéndices

1. INTRODUCCIÓN Y CONTEXTO

1.1 Origen del Proyecto

El estudio ficticio CHOAM Distributed Entertainment, especializado en simuladores estratégicos y mundos persistentes, ha identificado una oportunidad de mercado para una experiencia de gestión ambientada en el universo Dune. A diferencia de videojuegos monolíticos convencionales, la empresa desea desarrollar un prototipo técnicamente diseñado para funcionar en un contexto distribuido, demostrando competencias avanzadas de arquitectura de software.

La práctica académica que origina este proyecto consolida competencias de programación orientada a objetos, estructuras de datos, persistencia, manejo de excepciones, modelado de dominio y, de manera específica, diseño de software para entornos distribuidos.

1.2 Objetivos de Aprendizaje

El desarrollo de este proyecto permite al estudiante demostrar proficiency en las siguientes áreas competenciales:

La programación orientada a objetos se ejerce mediante el diseño de entidades con responsabilidades claras, herencia donde resulta apropiada, polimorfismo para comportamientos extensibles y encapsulamiento para proteger invariantes de negocio. El modelado de dominio requiere traducir requisitos narrativos en modelos de software que capturen la semántica del universo Dune mientras mantienen coherencia interna.

Las estructuras de datos se seleccionan considerando los patrones de acceso del sistema, optimizando para las operaciones más frecuentes mientras se mantiene claridad conceptual. La persistencia exige serialización confiable, manejo de versiones de esquema y recuperación ante fallos.

El manejo de excepciones en sistemas distribuidos presenta complejidades adicionales: fallos de red, timeouts, inconsistencia transitoria y datos obsoletos requieren estrategias específicas que el estudiante debe diseñar e implementar.

Finalmente, el diseño de software distribuido implica comprender patrones de comunicación, estrategias de consistencia, mecanismos de coordinación entre procesos y trade-offs entre disponibilidad y corrección.

1.3 Contexto Técnico

Desde la perspectiva técnica, la documentación de Microsoft recuerda que cuando una aplicación crece, conviene separar responsabilidades en capas lógicas y distinguirlas del despliegue físico por procesos o nodos. Esta separación es especialmente relevante en aplicaciones distribuidas, donde la topología de ejecución no coincide necesariamente con la organización interna del código.

Una solución puede poseer varias capas lógicas y, al mismo tiempo, desplegarse en uno o varios nodos físicos. Por ejemplo, una aplicación puede organizarse en capas de presentación, lógica de negocio y datos mientras se despliega en un cliente pesado, un servidor de aplicaciones y una base de datos separada, cada uno potencialmente ejecutándose en máquinas diferentes.

El modelo de comunicación entre componentes distribuidos puede ser síncrono o asíncrono. En interacciones síncronas, un componente solicita una operación y espera respuesta antes de continuar. En interacciones asíncronas, el emisor no bloquea su progreso esperando inmediatamente el resultado, permitiendo mayor throughput pero aumentando la complejidad del manejo de errores.

1.4 Alcance del Documento

Este documento define todos los aspectos del diseño del juego, desde la narrativa y mecánicas hasta la arquitectura técnica y plan de implementación. El alcance incluye los requisitos funcionales completos, las fórmulas matemáticas que gobiernan la simulación, el diseño de la arquitectura distribuida, las estructuras de datos seleccionadas y los algoritmos principales.

El documento no incluye assets gráficos más allá de referencias visuales para el equipo de arte, ya que la práctica se ejecuta en consola. Tampoco incluye el diseño de audio,音效 o música, elementos que quedarían pendientes para fases posteriores del desarrollo.

2. LORE Y AMBIENTACIÓN

2.1 El Universo Dune

Dune, la saga literaria creada por Frank Herbert y publicada originalmente en 1965, presenta un futuro distante donde la humanidad ha rechazado la dependencia de las máquinas pensantes tras una rebelión conocida como el Gjrat. Este rechazo de la computación mecánica llevó al desarrollo de las llamadas Mentats, humanos entrenados para realizar cálculos complejos, y de organizaciones como el Bene Gesserit, una hermandad femenina con habilidades físicas y mentales sobrehumanas.

La sociedad humana se organiza bajo un sistema feudal interestelar dominado por el Imperio Padishah, gobernado por Shaddam IV, Emperor de las Casas Combntantes. El poder imperial se sustenta en las Legiones Sardaukar, soldados de élite reclutados del planeta Salusa Secundus, combinado con alianzas con las Grandes Casas del Landsraad y el control sobre el recurso más valioso del universo conocido: la Melange, comúnmente llamada especia.

2.2 La Especia Melange

La Melange es una sustancia psicoactiva encontrada únicamente en el planeta Arrakis. Su consumo regular prolonga la vida, incrementa la vitalidad y expande la consciencia. Para los Navegantes de la Gilda, la especia permite la navegación interestelar segura, indispensable para el comercio y la comunicación en el Imperio. Para el Bene Gesserit, sus propiedades permiten ampliar la memoria genética y ejecutar el Proyecto Kwisatz Haderach.

La especia genera adicción física irreversible. Quien ha consumido Melange regularmente necesita dosis crecientes para funcionar, y la abstinencia es fatal. Esta dependencia universal asegura demanda perpetua y precios extraordinarios, convirtiendo el control de Arrakis en objetivo estratégico para todas las facciones del Imperio.

El ciclo vital de la especia está intrínsecamente ligado a los gusanos de arena de Arrakis. Las formaciones de pre-especia emergen bajo la arena en condiciones específicas de humedad y temperatura. Cuando estas formaciones alcanzan un punto crítico, los gusanos jóvenes las consumen y excretan Melange enriqueecida. El descubrimiento y extracción de estos afloramientos constituye la base de la economía arrakisiana.

2.3 Organizaciones del Universo

CHOAM

El Cuerpo de Manipulación Económica de Arrakis, conocido por su acrónimo CHOAM, representa el poder económico del universo Dune. Fundada como compañía monopolística para el comercio interstellar, CHOAM controla la producción, distribución y precios de prácticamente todos los bienes valiosos, incluyendo la especia.

Las acciones de CHOAM se distribuyen entre el Emperor, las Grandes Casas y la Gilda, con proporciones determinadas por tratados y guerras históricas. El Directorio de CHOAM ejecuta las operaciones diarias, pero las decisiones estratégicas requieren consenso entre los accionistas principales.

En el contexto del juego, el estudio ficticio CHOAM Distributed Entertainment toma su nombre de esta organización, apropiándose de su autoridad percibida para justificar la operación del jugador como subcontratista de facto.

La Casa Atreides

La Casa Atreides, históricamente establecida en el planeta oceánico Caladan, representa el arquetipo de nobleza honorable en el universo Dune. Bajo la guía del Duque Leto Atreides I, la Casa desarrolló tradiciones de gobierno justo y respeto por sus súbditos, contrastando con la crueldad de sus rivales, especialmente la Casa Harkonnen.

La asociación de Caladan con los Atreides influye en el escenario del mismo nombre disponible en el juego, proporcionando mejores condiciones logísticas y un tono más favorable para las operaciones del jugador.

Bene Tleilax

Los Tleilaxu son una civilizaciónhumanidad oculta en el planeta Tleilax, conocida por su dominio de la ingeniería genética y la producción de especia sintética. Aunque tecnológicamente capaces de replicar la Melange, su producto es considerado inferior y su uso está estigmatizado.

En el juego, las instalaciones de Bene Tleilax aceptan el descarte de criaturas problemáticas, cobrando una tarifa estándar. Esta mecánica permite al jugador deshacerse de ejemplares que consumen demasiados recursos o cuyo mantenimiento resulta económicamente inviable.

2.4 Escenarios Disponibles

El juego ofrece tres escenarios iniciales que determinan las condiciones de partida del jugador. Cada escenario presenta ventajas y desventajas específicas, orientando la estrategia inicial.

Arrakeen: Dominio de la Especia

El escenario principal sitúa al jugador en las instalaciones de Arrakeen, la capital administrativa de Arrakis. Esta ciudad emerge del desierto como testimonio del ingenio humano en el planeta más hostil del universo.

Arrakeen representa la operación más prestigiosa pero también la más compleja. El nivel adquisitivo ALTO de los visitantes potenciales maximiza el potencial de donations, pero las condiciones del desierto extremo requieren mayor inversión en instalaciones especializadas y suministros.

El jugador comienza con 100,000 Solaris, una cantidad sustancial que permite establecer una operación ambiciosa desde el principio. El enclave de exhibición Arrakeen ofrece 500 hectáreas y atrae 2,000 visitantes mensuales con nivel adquisitivo

ALTO.

Giedi Prime: Galería Industrial

El escenario alternativo de Giedi Prime transporta las operaciones a la luna artificial de Ix, donde la estética industrial y la eficiencia mecánica dominan el ambiente. Este escenario refleja las operaciones de la Casa Harkonnen, caracterizadas por ambientes controlados y visitantes de menor poder adquisitivo.

Giedi Prime ofrece menor riesgo inicial con 50,000 Solaris, pero las donaciones potenciales son significativamente menores debido al nivel adquisitivo BAJO. La estrategia óptima para este escenario enfatiza la eficiencia operativa sobre la maximización de ingresos por visitante.

El enclave de exhibición Giedi Prime ocupa 100 hectáreas con capacidad para 2,000 visitantes mensuales, pero el multiplicador σ de 1 reduce drásticamente los ingresos por donaciones comparado con Arrakeen.

Caladan: Reserva Ducal

El tercer escenario se desarrolla en el mundo oceánico de Caladan, asociado históricamente con la Casa Atreides. Este ambiente ofrece las condiciones más favorables para las operaciones, con mayor inversión inicial y mejores perspectivas de crecimiento.

Caladan proporciona 150,000 Solaris iniciales, el mayor capital de los tres escenarios. El nivel adquisitivo MEDIO de los 3,000 visitantes mensuales ofrece un balance entre riesgo y recompensa, mientras que las 10,000 hectáreas disponibles permiten expansión significativa.

Este escenario resulta ideal para jugadores que desean experimentar con sistemas de繁殖 sin las presiones económicas de Arrakis o la austeridad de Giedi Prime.

---

3. ANÁLISIS DE REQUISITOS

3.1 Requisitos Funcionales

El sistema debe permitir al jugador realizar operaciones de gestión en un entorno simulado con persistencia y trazabilidad. Los requisitos funcionales se organizan en categorías que reflejan las responsabilidades del sistema.

Gestión de Entidades

El sistema debe manejar las siguientes entidades con ciclos de vida completos: partidas con identificadores únicos y estado persistente, enclaves con capacidades de almacenamiento y atributos específicos, instalaciones con ocupaciones espaciales y restricciones de compatibilidad, criaturas con atributos dinámicos que cambian cada ronda, visitantes que llegan y abandonan enclaves según fórmulas definidas, y eventos que registran todas las acciones significativas.

Cada entidad requiere un identificador único que permita referenciarla sin ambigüedad, facilitar el debugging y soportar la serialización para persistencia.

Operaciones de Construcción

El sistema debe validar y ejecutar construcciones de instalaciones dentro de enclaves. Cada construcción implica verificación de fondos suficientes, disponibilidad de espacio en el enclave, compatibilidad del enclave con el tipo de instalación, y actualización del estado del enclave y la partida.

La construcción de una instalaciónded ACLIMATACIÓN o EXHIBICIÓN debe tener en cuenta las restricciones de medio y alimentación. Por ejemplo, una instalación de medio DESIERTO solo puede albergar criaturas adaptadas a ese ambiente.

Operaciones de Traslado

El sistema debe permitir mover criaturas adultas entre instalaciones. Un traslado requiere que la criatura origen haya alcanzado la edad adulta, mantenga salud suficiente, la instalación destino tenga capacidad libre, y el jugador disponga de fondos para el coste del traslado.

Los traslados fallidos deben reportarse con razones específicas que permitan al jugador corregir la situación sin repetir intentos costosos.

Simulación de Rondas

El sistema debe ejecutar simulaciones mensuales que procesen alimentación de criaturas, cambios de salud, nacimientos en instalaciones de aclimatación, llegada y abandono de visitantes, generación de donaciones, y actualización de edades de criaturas.

Cada ronda genera un conjunto de eventos que se almacenan cronológicamente, permitiendo al jugador revisar el histórico completo de su partida.

3.2 Requisitos de Arquitectura

La arquitectura debe demostrar separación clara de responsabilidades entre componentes. No se exige una implementación distribuida real con procesos separados, pero sí una organización técnica coherente que permita evolucionar hacia servicios independientes.

Modelo Compartido

Un proyecto de biblioteca de clases debe contener todas las entidades, enumerados, DTOs y contratos de interfaz utilizados por los demás componentes. Este proyecto no tiene dependencias hacia otros proyectos de la solución, permitiendo su reutilización universal.

Servicio de Persistencia

El servicio de persistencia es responsable de guardar y cargar partidas. Puede implementarse mediante serialización JSON, sistema de archivos estructurado, o base de datos local. La implementación debe ser intercambiable sin afectar otros componentes.

Servicio de Simulación

El servicio de simulación ejecuta la lógica de juego: cálculo de producciones, validación de operaciones, procesamiento de rondas. Este servicio consume el modelo compartido y puede comunicarse con el servicio de persistencia para guardar estado intermedio.

Cliente de Administración

El cliente de administración proporciona la interfaz de usuario, ya sea consola interactiva o interfaz gráfica ligera. El cliente traduce acciones del usuario en llamadas al servicio de simulación y presenta los resultados de manera comprensible.

3.3 Requisitos de Robustez

El sistema debe manejar gracefully situaciones de error sin perder datos ni entrar en estados inconsistentes.

Validación de Entrada

Toda operación debe validar sus parámetros antes de ejecutarse. Operaciones inválidas deben rechazar con mensajes informativos sin modificar el estado del sistema.

Recuperación de Errores

Errores de comunicación entre componentes deben capturarse, registrarse y comunicarse al usuario. El sistema debe definir políticas de reintento para operaciones transitorias y estrategias de rollback para operaciones compuestas.

Integridad de Estado

Tras cualquier operación, el sistema debe verificar que las invariantes se mantienen. Criaturas referenciadas deben existir en instalaciones válidas, las sumas de capacidad deben coincidir, y los recursos no deben ser negativos.

4. DISEÑO DEL SISTEMA DE ENCLAVES

4.1 Concepto de Enclave

Un enclave constituye una unidad territorial con gestión propia dentro del universo del juego. Cada enclave define un espacio físico con capacidades limitadas, recursos almacenables y potencial de atracción de visitantes.

Los enclaves se dividen en dos categorías funcionales: los de aclimatación, dedicados a la cría y mantenimiento inicial de criaturas, y los de exhibición, optimizados para atraer visitantes y generar ingresos mediante donaciones.

La separación entre tipos de enclaves refleja una decisión de diseño que permite a los jugadores enfocarse en diferentes aspectos de la gestión. Los enclaves de aclimatación requieren menor inversión inicial pero generan cero ingresos directos. Los enclaves de exhibición demandan mayor capital y gestión de visitantes, pero producen donations proporcionales al atractivo de las criaturas exhibidas.

4.2 Atributos de los Enclaves

Cada enclave posee un conjunto de atributos que determinan sus capacidades y comportamiento en el sistema.

El identificador único permite referenciar enclaves sin ambigüedad en operaciones de traslado y consulta. El nombre proporciona contexto humano para interfaces de usuario y registros de eventos. El tipo de enclave determina qué operaciones son válidas y cómo se calculan ciertos parámetros.

Las hectáreas representan el espacio físico disponible. Este atributo limita la cantidad total de instalaciones que pueden construirse. Las hectáreas ocupadas registran cuánto espacio utilizan las instalaciones actuales, mientras que las hectáreas disponibles calculan la diferencia.

Los suministros representan recursos materiales almacenados para alimentar criaturas. El almacén general del enclave mantiene suministros disponibles para distribución a instalaciones. La capacidad máxima de suministros equivale a las hectáreas del enclave multiplicadas por tres, reflejando restricciones de almacenamiento.

Los visitantes solo existen en enclaves de exhibición. Este atributo registra el número actual de visitantes presentes, queflucta cada ronda según las fórmulas de llegada y abandono.

El nivel adquisitivo determina el multiplicador sigma utilizado en cálculos de donación. Los niveles BAJO, MEDIO y ALTO corresponden a sigma igual a uno, quince y treinta respectivamente.

4.3 Configuración de Enclaves Iniciales

El diseño inicial del juego define cuatro enclaves preconfigurados disponibles según el escenario seleccionado.

La Cuenca Experimental de Arrakis representa el enclave de aclimatación común a todos los escenarios. Con mil hectáreas disponibles y suministros iniciales de cinco mil unidades, proporciona espacio suficiente para establecer programas de cría antes de expandirse a enclaves de exhibición. Este enclave no genera visitantes ni donations, enfocando todos los recursos en el desarrollo de criaturas de calidad.

Arrakeen, disponible únicamente en el escenario del mismo nombre, ofrece quinientas hectáreas con enclave de exhibición. Los dos mil visitantes mensuales con nivel adquisitivo ALTO lo convierten en la fuente de ingresos más rentable del juego, aunque las condiciones desérticas extremas elevan los costes operativos.

Giedi Prime, disponible en su escenario específico, presenta la operación más compacta con solo cien hectáreas. Los dos mil visitantes de nivel BAJO generan donations modestas, pero los costes reducidos permiten operaciones rentable para jugadores conservadores.

Caladan proporciona el enclave de exhibición más espacioso con diez mil hectáreas. Los tres mil visitantes de nivel MEDIO y la infraestructura oceánica ofrecen el mejor balance entre ingresos y costes operativos, resultando ideal para experimentación y aprendizaje.

4.4 Gestión de Suministros

Los suministros representan el recurso material necesario para alimentar criaturas. Su gestión eficiente impacta directamente en la salud de las criaturas y, consecuentemente, en los ingresos por donations.

El movimiento de suministros desde el almacén general del enclave hacia instalaciones individuales no genera coste adicional. Esta decisión de diseño simplifica la gestión diaria, permitiendo al jugador redistribuir recursos libremente según necesidades cambiantes.

La restricción de capacidad establece que ninguna instalación puede almacenar más suministros que su coste de construcción. Esta regla previene estrategias de acumulación excesiva que trivializarían la gestión de recursos. Una instalación ADR05 con coste de mil Solaris puede almacenar máximo mil unidades de suministros, equivalentes a aproximadamente doscientas criaturas-día para un gusano de arena adulto.

La compra de suministros adicionales tiene un coste fijo de cinco Solaris por unidad. Esta simplicidad permite proyecciones financieras claras sin variables adicionales complicando el análisis.

---

5. DISEÑO DEL SISTEMA DE CRIATURAS

5.1 Filosofía del Sistema de Criaturas

El sistema de criaturas captura la biodiversidad de Arrakis y sus mundos vecinos, permitiendo a los jugadores experimentar con la domesticación, cría y exhibición de fauna alienígena. Cada especie presenta características únicas que influyen en su valor comercial, requisitos de alimentación y viabilidad en diferentes tipos de instalaciones.

El diseño equilibra realismo biológico simplificado con accesibilidad lúdica. Las criaturas envejecen, se alimentan, enferman y eventualmente mueren, creando consecuencias tangibles para las decisiones del jugador. Sin embargo, los ciclos de vida se comprimenen meses jugables, evitando la frustración de esperar décadas para ver resultados.

5.2 Atributos de las Criaturas

Cada criatura posee un conjunto de atributos que definen su estado actual y comportamiento en la simulación.

El identificador único permite tracking individual de cada ejemplar, esencial para registro de eventos y trazabilidad genealógica. La especie determina los valores base de medio, alimentación, edad adulta y apetito. El medio indica el tipo de ambiente al que la

criatura está adaptada: DESIERTO para especies de superficie tolerantes al calor, AÉREO para criaturas voladoras que requieren instalaciones con cielos abiertos, y SUBTERRÁNEO para especies adaptadas a espacios profundos como los gusanos de arena.

La alimentación define el tipo de recursos que la criatura consume: RECOLECTOR para especies que obtienen energía de fuentes vegetales o minerales disponibles en su ambiente, y DEPREDADOR para especies que requieren alimentarse de otras criaturas o materia orgánica compleja.

La edad registra los meses transcurridos desde el nacimiento de la criatura. Este valor determina si la criatura es adulta según el umbral especie-específico, influye en los requisitos de alimentación, y afecta el atractivo visual para visitantes mediante el factor de edad en el cálculo de donations.

La salud es un porcentaje entre cero y cien que indica el bienestar físico de la criatura. Valores bajos reducen el atractivo para visitantes, incrementan el riesgo de muerte, y eventualmente causan letargo irreversible cuando alcanzan cero.

El contador de veces favorita registra cuántas veces visitantes han elegido esta criatura como su favorita. Este atributo influye en el prestige del jugador y proporciona feedback sobre qué ejemplares resultan más atractivos.

5.3 Catálogo de Especies

Gusano de Arena Juvenil (Shai-Hulud)

El Gusano de Arena representa la fauna más icónica de Arrakis y el universo Dune en general. Los ejemplares juveniles presentes en el juego son versiones pre-adultas del Shai-Hulud completo, cuyo tamaño maduro alcanzaría kilómetros de longitud.

Este especímen es subterráneo con alimentación depredadora. Su edad adulta se alcanza a los veinticuatro meses, moderadamente rápida comparada con otras especies. El apetito base de cinco unidades lo convierte en un consumidor eficiente, aunque los ejemplares maduros aumentan significativamente sus demandas alimentarias.

El gusano de arena genera donations sustanciales debido a su atractivo legendario. Un ejemplar sano y adulto puede acumular donaciones significativas durante su vida útil, compensando los altos costes de alimentación en instalaciones subterráneas.

El coste de traslado para gusanos de arena es elevado debido a su clasificación subterránea. Con sigma de veinticinco, mover un ejemplar adulto cuesta dos mil quinientos Solaris, pero ejemplares más viejos pueden alcanzar costes prohibitivos debido al factor exponencial basado en edad.

Tigre Laza

El Tigre Laza es un depredador del desierto de tamaño medio, reconocible por sus marcas distintivas que evocan la silueta del universo. Su naturaleza depredadora requiere instalaciones que puedan proporcionar alimentos de manera sostenible.

Con treinta y ocho meses hasta la edad adulta, el Tigre Laza presenta uno de los ciclos de desarrollo más largos entre las especies disponibles. Su apetito base de ocho unidades genera costes operativos significativos, especialmente cuando múltiples ejemplares maduros compiten por recursos limitados.

El Tigre Laza ofrece moderate donations debido a su atractivo visual para visitantes, pero la inversión a largo plazo en alimentación puede superar los ingresos generados antes de que el ejemplar alcance su máximo potencial comercial.

Muad'Dib

Muad'Dib, nombre que los Fremen dan a la liebre del desierto, representa una especie adaptada a la vida en dunas arenosas. Su alimentación recolectora lo hace relativamente económico de mantener comparado con depredadores.

La madurez rápida a los doce meses permite ciclos de cría acelerados. El appetite base de solo dos unidades minimiza los costes de operación, mientras que la naturaleza recolectora elimina la necesidad de provechar otras criaturas como alimento.

Aunque las donations individuales resultan modestas, la combinación de bajos costes operativos y madurez rápida hace del Muad'Dib una especie excelente para establecer programas de cría rentables.

Halcón del Desierto

El Halcón del Desierto es un ave rapaz adaptada a las corrientes térmicas de Arrakis. Su medio aéreo requiere instalaciones especiales con espacios abiertos, típicamente implementados como cúpulas blindadas.

La madurez a los dieciséis meses y el appetite base de dos unidades ofrecen un balance atractivo entre inversión temporal y costes operativos. El coste de traslado moderado con sigma de quince lo hace accesible para programas de exhibición distribuidos.

Trucha de Arena

La Trucha de Arena es una especie submarina adaptada a los océanos subterráneos de Arrakis. Sorprendentemente, esta criatura vive en ambientes subterráneos pero se alimenta de recursos recolectados, creando una combinación única de requisitos de instalación.

Con cuarenta y dos meses hasta la madurez, la Trucha de Arena presenta el ciclo de desarrollo más largo del catálogo. Sin embargo, su appetite base de diez unidades lo convierte en un consumidor eficiente a largo plazo. El alto coste de traslado con sigma de veinticinco refleja las dificultades logísticas de mover especímenes subterráneos.

5.4 Sistema de Alimentación

La alimentación constituye la mecánica central de supervivencia de las criaturas. Cada ronda, el sistema calcula los requisitos alimentarios de cada ejemplar y distribuye suministros disponibles desde las instalaciones que los albergan.

Para criaturas pre-adultas, la cantidad requerida de alimentos se calcula multiplicando el appetite base por la edad actual. Un Muad'Dib de seis meses requiere doce unidades de suministros mensuales. Esta fórmula lineal reconoce que los ejemplares jóvenes tienen necesidades proporcionales a su desarrollo.

Para criaturas adultas, la fórmula cambia a appetite base multiplicado por dos elevado a la edad menos la edad adulta, todo multiplicado por un factor alfa. Este factor alfa vale uno en instalaciones de exhibición y quince en instalaciones de aclimatación, reflejando las diferentes prioridades de cada tipo de instalación.

La implementación de este cálculo requiere atención cuidadosa a los tipos de datos. La edad puede alcanzar valores donde la función exponencial genere números extremadamente grandes, potencialmente causando overflow. El diseño debe establecer límites razonables o implementar saturación aritmética.

5.5 Sistema de Salud

La salud de una criatura refleja su estado físico general, determinado por la calidad de su alimentación en rondas recientes. Una criatura bien alimentada mantiene o recupera salud, mientras que la alimentación insuficiente causa deterioro progresivo.

Si una criatura recibe menos del veinticinco por ciento de su ingesta requerida, pierde treinta unidades de salud. Este penalty severocastiga la negligencia grave y requiere intervención inmediata para evitar pérdidas.

Si la criatura recibe entre el veinticinco y el setenta y cinco por ciento de lo requerido, pierde veinte unidades de salud. Este penalty moderado reconoce dificultades parciales de gestión sin ser inmediatamente fatal.

Si la criatura recibe más del setenta y cinco por ciento pero menos del cien por ciento, pierde diez unidades de salud. Este penalty menor refleja que la subalimentación crónica, aunque no crítica, eventualmente afecta el bienestar.

Si la criatura recibe su ingesta completa y su salud es menor a cien, recupera cinco unidades de salud. Este bonus de recuperación incentiva la gestión cuidadosa y permite restaurar criaturas dañadas.

Cuando la salud de una criatura alcanza cero, entra en estado de letargo irreversible. El letargo representa muerte clínica: la criatura no puede recuperarse y debe ser descartada mediante transferencia a instalaciones de Bene Tleilax por un coste fijo de veinte mil Solaris.

5.6 Reproducción

Las instalaciones de aclimatación pueden generar nuevas criaturas mediante reproducción o clonación controlada. Este mecanismo proporciona una fuente continua de ejemplares sin depender de capturar criaturas salvajes, que quedan fuera del alcance del diseño actual.

Para que ocurra un nacimiento, la instalación debe tener capacidad libre para alojar un nuevo ejemplar. Si la instalación está completa, no se intenta reproducción hasta que un occupant sea trasladado, descartado, o muera.

La especie del nuevo ejemplar se selecciona aleatoriamente entre todas las especies disponibles, sin consideración por las especies presentes en la instalación o el enclave. Esta decisión de diseño introduce variedad genética artificial que evita la especialización excesiva.

La probabilidad de nacimiento exitoso es del veinte por ciento por ronda. Esta tasa relativamente baja balancea la necesidad de generar nuevos ejemplares con la presión de gestionar una población creciente. Un enclave con múltiples instalaciones puede esperar varios nacimientos mensuales en promedio, pero la varianza introduce incertidumbre estratégica.

Los ejemplares nacidos comienzan con edad cero, salud de cien, y cero veces favorita. Su medio y alimentación se determinan según su especie. Deben alcanzar la edad adulta antes de poder ser trasladados a instalaciones de exhibición.

6. DISEÑO DEL SISTEMA DE INSTALACIONES

6.1 Arquitectura de Instalaciones

Las instalaciones constituyen las estructuras construibles dentro de los enclaves, cada una diseñada para propósitos específicos de aclimatación o exhibición. Una instalación define qué criaturas puede albergar según su medio y patrón de alimentación, cuánta capacidad de alojamiento proporciona, cuánto espacio físico ocupa, y qué recursos necesita para funcionar.

El diseño de instalaciones equilibra especialización con flexibilidad. Instalaciones especializadas ofrecen mayor capacidad por hectarea pero restringen qué criaturas pueden usarlas. Instalaciones generalistas admiten más tipos de criaturas pero típicamente con menor eficiencia.

6.2 Instalaciones de Aclimatación

Las instalaciones de aclimataciónpriorizan la salud y reproducción de criaturas sobre el atractivo visual. Su diseño refleja las condiciones controladas necesarias para criar especies delicadas.

La instalación ADR05 es la opción más básica de aclimatación. Con un coste de mil Solaris y ocupación de diez hectáreas, puede albergar hasta cinco criaturas de medio DESIERTO con alimentación RECOLECTORA. Los suministros iniciales de doscientas unidades permiten mantener varios ejemplares durante varias rondas sin reposición. El recinto de roca sellada proporciona protección contra elementos externos mientras mantiene las condiciones de desert que estas criaturas requieren.

La instalación ADP03 duplica el espacio a cincuenta hectáreas a cambio de mayor capacidad de criaturas, hasta tres ejemplares. El coste elevado de dos mil quinientos Solaris refleja la infraestructura adicional necesaria para depredadores del desierto, que requieren compartimentos de alimentación separados y sistemas de seguridad más robustos.

La instalación AAV02 representa la única opción para criaturas de medio AÉREO. El coste de cinco mil Solaris y las cien hectáreas de ocupación limitan su viabilidad económica, pero resulta indispensable para criar halcones del desierto. La cúpula

blindada proporciona el cielo abierto necesario mientras protege a las criaturas del clima extremo.

La instalación ASU04 atiende criaturas subterráneas. Con cuatrocientos Solaris de coste y veinticinco hectáreas, ofrece la mejor proporción de capacidad por inversión entre las instalaciones de aclimatación. El pozo reforzado proporciona el ambiente profundo que gusanos de arena y truchas requieren.

6.3 Instalaciones de Exhibición

Las instalaciones de exhibiciónpriorizan la experiencia del visitante sobre la funcionalidad de cría. Su diseño maximiza el atractivo visual mientras mantiene condiciones aceptables para las criaturas albergadas.

La instalación EDR02 aloja criaturas de desierto recolectoras en recintos de roca sellada. Con doscientos Solaris de coste y doscientos Solaris de ocupación, es la instalación de exhibición más accesible. Sin embargo, su capacidad de solo dos criaturas limita el potencial de generación de donations.

La instalación EDP03 presenta un enigma económico: con apenas mil doscientos cincuenta Solaris de coste y trescientos Solaris de ocupación, es la instalación más barata del juego. Sin embargo, su capacidad de solo tres criaturas de desierto depredadoras puede no justificar la ocupación de espacio valuable en enclaves de exhibición prime.

La instalación EAV02 para criaturas aéreas requiere quince mil Solaris y doscientos hectáreas. La cúpula blindada proporciona visibilidad óptima para visitantes mientras mantiene condiciones habitables para halcones. La capacidad de dos criaturas limita el retorno de la inversión, pero el atractivo único de las criaturas aéreas justifica su inclusión.

La instalación ESU03 para criaturas subterráneas representa la inversión más significativa con veinticinco mil Solaris y cuatrocientos Solaris de ocupación. La capacidad de tres criaturas justifica parcialmente el coste cuando se exhiben gusanos de arena adultos, cuyo atractivo legendary genera donations sustanciales.

6.4 Mecánicas de Funcionamiento

Proceso de Alimentación

Cada ronda, el sistema procesa alimentación para todas las criaturas en todas las instalaciones. El algoritmo itera sobre instalaciones, luego sobre criaturas dentro de cada instalación.

Para cada criatura, el sistema calcula la ingesta requerida según las fórmulas documentadas anteriormente. Luego determina cuántos suministros tiene disponibles la instalación, distribuyendo proporcionalmente si los suministros son insuficientes.

Los suministros consumidos se restan del stock de la instalación. Criaturas que no reciben alimentación completa sufren penalties de salud según el porcentaje de requerimiento no satisfecho.

Monetización

En instalaciones de exhibición, el sistema procesa la elección de criaturas favoritas por parte de los visitantes. Cada visitante tiene una probabilidad de elegir una criatura como favorita, típicamente diez por ciento.

Cuando un visitante elige una favorita, la donación se calcula según la fórmula de donations y se añade al total de Solaris del jugador. La criatura incrementasu contador de veces favorita.

Cálculo de Producción Neta

Cada instalación tiene un coste operativo implícito derivado del mantenimiento de condiciones, seguridad y personal. Este coste no se descuenta directamente, sino que se refleja en la necesidad de mantener suministros comprados. El jugador debe generar suficientes donations para justificar la inversión en construcción y operación de cada instalación.

7. SISTEMA DE VISITANTES

7.1 Concepto de Sistema de Visitantes

Los visitantes representan el flujo de público que llega a los enclaves de exhibición, generando ingresos mediante donations cuando eligen criaturas como favoritas. El sistema de visitantes captura la relación entre el atractivo del enclave, la calidad de las criaturas exhibidas, y los ingresos generados.

El diseño asume que los visitantes llegan atraídos por la reputación del enclave, permanecen durante su visita explorando las instalaciones y observando criaturas, y eventualmente abandonan el enclave después de un período de estancia que depende de su satisfacción.

7.2 Cálculo de Llegada de Visitantes

Cada ronda, el sistema calcula cuántos visitantes llegan al enclave basándose en parámetros fijos del enclave y el estado dinámico de las criaturas.

La fórmula de llegada multiplica el número base de visitantes por la proporción de hectáreas utilizadas por instalaciones, y luego por la salud media de las criaturas del enclave. Esta fórmula refleja que enclaves más desarrollados con criaturas saludables atraen más visitantes.

El factor de hectáreas utilizadas normaliza contra el espacio disponible. Un enclave con todas sus hectáreas ocupadas atraerá más visitantes que uno con instalaciones dispersas, asumiendo que más instalaciones significa más criaturas para observar.

El factor de salud media reconoce que visitantes evitandenclaves donde las criaturas lucen enfermas o moribundas. Criaturas en letargo o con salud críticamente baja reducen drásticamente el atractivo del enclave.

7.3 Cálculo de Abandono de Visitantes

Simultáneamente, visitantes existentes deciden si permanecen o abandonan el enclave. La fórmula de abandono usa la misma estructura que la de llegada, generando un número de abandonos que se restan del total de visitantes actuales.

El equilibrio entre llegadas y abandonos determina si la población de visitantes crece, decrece, o se mantiene estable. Enclaves prósperos con criaturas saludables experimentan crecimiento poblacional, mientras que enclaves negligenciados pierden visitantes progresivamente.

7.4 Donaciones por Criatura Favorita

Cuando un visitante elige una criatura como favorita, el jugador recibe una donación en Solaris. Esta mecánica representa el pago discrecional de visitantes particularmente impresionados por una criatura específica.

La fórmula de donación multiplica un factor base de diez por la proporción de salud de la criatura, la proporción de su edad respecto a la edad adulta, y el multiplicador sigma del enclave. Criaturas sanas y maduras generan donations significativamente mayores que ejemplares jóvenes o enfermos.

Un gusano de arena adulto con salud completa en Arrakeen genera donaciones de aproximadamente trescientos Solaris por favorita. Si cien visitantes eligen al gusano como favorito durante una ronda, esto representa treinta mil Solaris de ingresos, sustancialmente más que el coste de alimentación de un gusano maduro.

7.5 Interacción con Sistema de Criaturas

El sistema de visitantes interactúa con el sistema de criaturas de varias maneras. Las donations generadas por criaturas favoritas representan ingresos que pueden usarse para mejorar instalaciones, comprar suministros, o costear traslados de más ejemplares.

Simultáneamente, la presencia de visitantes crea presión sobre la gestión de criaturas. Criaturas con baja salud generan menos donations y potencialmente alejan visitantes existentes, creando un ciclo negativo que puede espiral hacia la pérdida del enclave.

La decisión estratégica de qué criaturas exhibiry cuándo trasladar ejemplares adultos desde aclimatación a exhibición constituye el núcleo del gameplaydel juego. Los jugadores deben balancear el riesgo de mover criaturas prematuramente contra el costo de oportunidad de mantener criaturas improductivas en instalaciones de cría.

8. DISEÑO DEL SISTEMA DE RECURSOS

8.1 Tipos de Recursos

El juego implementa dos tipos principales de recursos que los jugadores deben gestionar: Solaris como moneda universal, y suministros como recurso material para alimentación.

Los Solaris representan el capital económico del jugador, usado para todas las transacciones del juego: construcción de instalaciones, traslado de criaturas, compra de suministros, y costes de descarte. Los Solaris se ganan mediante donations de visitantes y potencialmente mediante otras mecánicas futuras como venta de especias.

Los suministros representan recursos materiales almacenados en enclaves e instalaciones. Se consumen cada ronda para alimentar criaturas y pueden comprarse a un precio fijo. A diferencia de los Solaris, los suministros no pueden transferirse entre enclaves directamente; deben estar disponibles en la instalación donde se necesitan.

8.2 Flujo de Solaris

Fuentes de Ingreso

La fuente primaria de Solaris en el juego actual son las donations de visitantes en enclaves de exhibición. Cada donation se genera cuando un visitante elige una criatura como favorita, con el monto determinado por la fórmula documentada anteriormente.

Las donations representan ingresos pasivos derivados de mantener criaturas saludables y exhibiciones atractivas. El jugador no necesita acción específica para generar donations más allá de mantener el enclave operando correctamente.

Usos de Solaris

Los Solaris se gastan en construcción de instalaciones. Cada tipo de instalación tiene un coste de construcción que se descuenta inmediatamente al confirmar la operación. Fondos insuficientes causan rechazo de la construcción.

Los traslados de criaturas adultas entre instalaciones tienen un coste variable basado en la edad de la criatura y su medio. Criaturas más viejas son más costosas de mover debido al riesgo logístico.

El descarte de criaturas a Bene Tleilax tiene un coste fijo de veinte mil Solaris. Este precio elevado desincentiva el descarte casual y crea presión para mantener criaturas vivas o aceptar pérdidas significativas.

La compra de suministros cuesta cinco Solaris por unidad. Aunque parece modesto, los ejemplares adultos con alto appetite pueden requerir inversiones sustanciales para mantener saludables.

8.3 Flujo de Suministros

Los suministros fluyen desde el almacén general del enclave hacia las instalaciones según las necesidades de las criaturas alojadas. El jugador no controla este flujo manualmente; el sistema de simulación distribuye suministros automáticamente según las fórmulas de alimentación.

La compra de suministros aumenta el stock del almacén general del enclave. Esta operación no tiene coste variable por cantidad, pero está limitada por la capacidad de almacenamiento del enclave.

El almacenamiento tiene un límite de tres veces las hectáreas del enclave. Un enclave de quinientas hectáreas puede almacenar máximo mil quinientas unidades de suministros, obligando a reposiciones frecuentes si las criaturas tienen alto consumo.

8.4 Balance Económico

El diseño económico busca crear un ciclo virtuoso donde la gestión exitosa de criaturas genera donations, las donations financian mejoras, las mejoras atraen más visitantes, y los visitantes generan más donations.

Criaturas sanas y adultas en enclaves de nivel adquisitivo alto generan donations sustanciales. Un gusano de arena adulto bien mantenido en Arrakeen puede generar miles de Solaris mensuales, justificando la inversión en instalaciones subterráneas y suministros.

Criaturas enfermas o jóvenes generan donations mínimas o nulas mientras consumen recursos. Una poblaciónno gestionada puede drenar los fondos del jugador hasta la bancarrota, creando presión para decisiones de descarte o reubicación.

El diseño de los escenarios proporciona diferentes puntos de partida para este ciclo. Arrakeen ofrece alto potencial pero alto riesgo. Giedi Prime ofrece estabilidad pero crecimiento limitado. Caladan ofrece el mejor punto medio para experimentación.

9. SISTEMA DE RONDAS Y SIMULACIÓN

9.1 Concepto de Ronda

Una ronda representa un mes en el universo del juego, durante el cual ocurren todos los procesos de simulación: alimentación de criaturas, cambios de salud, nacimientos, llegada y abandono de visitantes, generación de donations, y actualización de atributos de edad.

El jugador puede ejecutar rondas manualmente cuando haya completado todas las decisiones de gestión deseadas, o puede delegar la ejecución continua de rondas al sistema con mínima intervención.

9.2 Estructura del Procesamiento de Ronda

El procesamiento de una ronda sigue una secuencia definida de fases, cada una actualizando diferentes aspectos del estado del juego.

La primera fase procesa nacimientos en instalaciones de aclimatación. Para cada instalación con espacio libre, el sistema genera un número aleatorio y, si está por debajo del umbral del veinte por ciento, crea una nueva criatura de especie aleatoria.

La segunda fase alimenta criaturas. El sistema itera sobre todas las instalaciones de todos los enclaves, calculando ingesta requerida para cada criatura, verificando disponibilidad de suministros, aplicando effects a la salud según el porcentaje de requerimiento satisfecho, y actualizando los stocks de suministros.

La tercera fase procesa eventos de muerte. Criaturas cuya salud alcanzó cero entran en letargo irreversible y se marcan para descarte obligatorio en la fase de resolución.

La cuarta fase calcula cambios de visitantes. Usando las fórmulas de llegada y abandono, el sistema actualiza el número de visitantes en cada enclave de exhibición.

La quinta fase procesa elecciones de favoritos y donations. Cada visitante tiene probabilidad de elegir una criatura como favorita; si la elección ocurre, se calcula y otorga la donation correspondiente.

La sexta fase incrementa la edad de todas las criaturas en un mes. Criaturas que cruzan el umbral de edad adulta se marcan como tales, habilitando su traslado a instalaciones de exhibición.

La séptima fase aplica descarte automático de criaturas en letargo, cobrando el coste de Bene Tleilax y registrando el evento.

La octava fase guarda el estado actualizado en persistencia, asegurando que el progreso no se pierde ante fallos del sistema.

9.3 Resultado de Ronda

Cada ronda procesada genera un resultado que incluye el número de mes, los eventos ocurridos durante la simulación, los cambios en recursos, y estadísticas agregadas.

Los eventos registrados incluyen nacimientos con detalles de especie y progenitor, muertes por letargo con información de la criatura afectada, donations con montos y criaturas involucradas, traslados completados, construcciones nuevas, y errores encontrados durante el procesamiento.

Los cambios en recursos detallan variaciones en Solaris, suministros por enclave, y cualquier otro activo del juego.

Las estadísticas agregadas proporcionan contexto sobre el rendimiento del enclave: visitantes totales, donations totales, criaturas totales, salud promedio, y otras métricas relevantes.

9.4 Condiciones de Fin de Partida

El juego puede terminar por varias razones que determinan el resultado final.

La victoria se define mediante un objetivo configurable. Por defecto, alcanzar cierto monto de Solaris o mantener un enclave operating durante un número de rondas sin quiebra constituye victoria.

La derrota ocurre cuando los Solaris del jugador se vuelven negativos, indicando bancarrota. En este estado, el jugador no puede realizar operaciones que requieran fondos y debe liquidar activos para sobrevivir.

El abandono permite al jugador terminar voluntariamente una partida, guardando el estado final y calculando estadísticas de rendimiento.

10. ARQUITECTURA DISTRIBUIDA

10.1 Principios de Diseño

La arquitectura distribuida del sistema refleja los principios de separación de responsabilidades, comunicación definida, y evolvabilidad. Aunque el prototipo puede ejecutarse en un único proceso, la organización del código debe demostrar conciencia de los desafíos distribuidos.

La separación de componentes permite desarrollo independiente de diferentes capas funcionales. El equipo de frontend puede trabajar en interfaces de usuario mientras el equipo de backend desarrolla lógica de simulación, siempre que respeten los contratos de interfaz definidos.

La comunicación explícita entre componentes fuerza la definición de APIs claras que documentan qué operaciones son posibles y qué resultados esperar. Esta documentación implícita facilita onboarding de nuevos desarrolladores y reduce acoplamientos accidentales.

La evolvabilidad hacia servicios separados se logra manteniendo los límites de proyecto incluso cuando todos los componentes ejecutan en el mismo proceso. El código debe escribirse como si los componentes estuvieran realmente en diferentes máquinas, permitiendo migración futura sin reescritura.

10.2 Modelo Compartido

El proyecto de modelo compartido constituye la base sobre la que construyen todos los demás componentes. Contiene las definiciones de entidades, enumerados, y contratos de interfaz que todos comparten.

Las entidades del modelo compartido incluyen Partida con su identificador único, alias del jugador, escenario, mes actual, Solaris, y referencias a enclaves. Enclave con tipo, hectáreas, suministros, visitantes, y colección de instalaciones. Instalacion con código, tipo, medio, alimentación, capacidad, costes, y criaturas alojadas. Criatura con especie, atributos dinámicos, y ubicación actual. Y Evento con timestamp, tipo, descripción, y datos específicos.

Los enumeradosdefinen los espacios de valores para tipos de enclave, niveles adquisitivos, especies, medios, alimentaciones, y tipos de evento. Usar enumerados en lugar de strings crudos mejora seguridad de tipos y permite completado de código en IDEs.

Los DTOs proporcionan estructuras para transferencia de datos entre componentes sin exponer modelos internos. Un DTO de Partida puede incluir solo los campos necesarios para mostrar información, omitiendo detalles de implementación que el cliente no necesita.

Los contratos de interfaz definen las operaciones disponibles entre componentes. IServicioSimulacion declara los métodos que el cliente puede invocar; IServicioPersistencia declara cómo se guardan y cargan datos.

10.3 Servicio de Simulación

El servicio de simulación encapsula toda la lógica de juego. Su responsabilidad principal es mantener el estado de la partida actualizado según las reglas del juego y ejecutar simulaciones de rondas.

El servicio recibe comandos del cliente para crear partidas, ejecutar operaciones, y procesar rondas. Valida los comandos según las reglas del juego, aplica los cambios al estado, y genera eventos que el cliente puede mostrar al usuario.

La implementación del servicio de simulación no conoce detalles de cómo se comunica con el cliente o cómo se persiste el estado. Esta separación permite que el mismo servicio opere en diferentes contextos: aplicación de consola, servicio web, o aplicación gráfica.

10.4 Servicio de Persistencia

El servicio de persistencia gestiona el almacenamiento y recuperación de datos del juego. Su responsabilidad es asegurar que el estado del juego sobreviva a reinicios del programa y pueda compartirse entre componentes si la arquitectura evoluciona a múltiples procesos.

La implementación puede usar serialización JSON a archivos, base de datos local, o cualquier otra tecnología que cumpla los requisitos de guardar partida, cargar partida, listar partidas disponibles, y eliminar partidas.

El servicio de persistencia no contiene lógica de juego. Solo sabe cómo serializar y deserializar estructuras de datos. Esta separación permite cambiar la tecnología de almacenamiento sin afectar otros componentes.

10.5 Cliente de Administración

El cliente de administración proporciona la interfaz de usuario al sistema. Puede ser una aplicación de consola interactiva, una interfaz gráfica, o cualquier otro medio de interacción.

El cliente traduce acciones del usuario en llamadas al servicio de simulación, esperando resultados y mostrándolos de manera comprensible. También consulta al servicio de persistencia para listar partidas guardadas y cargar estados previos.

Un cliente bien diseñado trata los servicios remotos como cajas negras que responden a comandos y queries. No debe asumir conocimiento de implementación interna de los servicios.

10.6 Patrones de Comunicación

Comunicación Síncrona

La comunicación síncrona implica que el cliente envía una petición y espera la respuesta antes de continuar. Este patrón es apropiado para operaciones que requieren confirmación inmediata: crear partida, ejecutar operación unitaria, consultar estado.

La implementación síncrona es más simple de entender y depurar. El flujo de control es lineal: usuario actúa, sistema responde, usuario observa resultado.

Comunicación Asíncrona

La comunicación asíncrona implica que el cliente envía una petición y continúa sin esperar respuesta. El resultado se entrega mediante callback o polling posterior. Este patrón es apropiado para operaciones de larga duración como procesamiento de múltiples rondas.

La implementación asíncrona permite mejor throughput cuando hay muchas operaciones independientes. El usuario puede iniciar múltiples acciones simultáneamente y el sistema las procesa en paralelo.

Sistema de Eventos

El sistema de eventos implementa el patrón publish-subscribe donde componentes registran interés en ciertos tipos de eventos y reciben notificaciones cuando ocurren. Este patrón desacopla emisores de receptores, permitiendo que el servicio de simulación notifique múltiples interesados sin conocer sus identidades.

Un logger puede suscribirse a todos los eventos para construir el registro de auditoría. Una interfaz gráfica puede suscribirse a eventos relevantes para actualizar la pantalla. Un sistema de achievements puede suscribirse a logros desbloqueables.

---

11. DISEÑO DEL CENTRO DE MANDO

11.1 Propósito del Centro de Mando

El Centro de Mando constituye la interfaz principal desde la que el jugador supervisa y gobierna el estado de su partida. Su diseño debe proporcionar información completa mientras mantiene claridad y facilidad de navegación.

Un Centro de Mando efectivo presenta información estructurada, permite drill-down desde vistas agregadas a detalles específicos, y proporciona acceso directo a operaciones comunes. El jugador debe poder responder preguntas como cuántas criaturas tengo, cuál necesita atención, dónde está el cuello de botella económico, y qué ha pasado en las últimas rondas.

11.2 Vista General de Partida

La vista general presenta el estado de alto nivel de la partida: alias del jugador, escenario actual, mes de juego, Solaris disponibles, y resúmenes de enclaves. Esta vista responde a la pregunta de conjunto qué está pasando sin entrar en detalles.

Los indicadores visuales muestran la salud general de cada enclave, alertando sobre situaciones que requieren atención. Un enclave con criaturas en estado crítico debería mostrarse en rojo; uno próspero, en verde.

La navegación desde esta vista permite acceso a cualquier enclave específico o pantalla de detalle.

11.3 Vista de Enclave

La vista de enclave presenta todos los detalles de un enclave específico: sus atributos, lista de instalaciones, criaturas alojadas, visitantes actuales, y recursos almacenados.

Las instalaciones se presentan con su código, tipo, ocupación actual versus capacidad, y salud agregada de criaturas. Instalaciones con espacio disponible se marcan claramente; las llenas muestran indicación visual.

Las criaturas dentro de cada instalación se ordenan decrecientemente por salud, mostrando primero las que más atención requieren. Cada entrada muestra especie, edad, salud actual, y veces que ha sido favorita.

Los visitantes se presentan como un número aggregate junto con estadísticas de tendencia. Si los visitantes están creciendo o decreciendo, el indicador de tendencia proporciona contexto.

11.4 Vista de Criatura Individual

La vista de criatura proporciona el máximo nivel de detalle para un espécimen específico. Todos los atributos se muestran junto con contexto histórico: tiempo como favorita, donations generadas, tratamientos médicos recibidos.

Las estadísticas de rendimiento permiten evaluar el valor de la criatura para el enclave. Un gusano de arena que ha generado diez mil Solaris en donations justifica su mantenimiento aunque sus costes operativos sean elevados.

Los controles disponibles muestran qué operaciones pueden realizarse con la criatura: trasladar si es adulta y tiene salud suficiente, descartar si es necesario, o ninguna si no cumple requisitos.

11.5 Registro de Eventos

El registro de eventos muestra el histórico completo de acciones en orden cronológico inverso, con los eventos más recientes primero. Cada entrada incluye timestamp, tipo de evento, descripción legible, y datos específicos relevantes.

Los eventos se agrupan por sesión cuando corresponde, facilitando la identificación de patrones. Si varios nacimientos ocurren en la misma ronda, se agrupan visualmente.

Los filtros permiten buscar eventos específicos: solo donaciones, solo muertes, solo traslados. Esta capacidad de filtrado facilita el análisis post-hoc de decisiones pasadas.

11.6 Consultas y Reporting

Más allá de vistas predefinidas, el Centro de Mando puede soportar consultas ad-hoc que respondan preguntas específicas del jugador.

Un reporte de profitability puede mostrar qué instalaciones generan más donations netas después de costes de alimentación. Esto ayuda al jugador a identificar qué inversiones valen la pena.

Un reporte de criaturas en riesgo puede listar todos los ejemplares con salud por debajo de cierto umbral, permitiendo intervención proactiva antes de pérdidas.

Un reporte de proyección puede estimar el estado futuro del enclave dados ciertos supuestos, ayudando al jugador a planificar inversiones.

---

12. GESTIÓN DE ERRORES Y ROBUSTEZ

12.1 Principios de Robustez

El sistema debe funcionar correctamente bajo condiciones adversas: entradas inválidas, fallos de componentes, y estados inconsistentes. La robustez no es opcional sino requisito fundamental.

La defensa en profundidad establece que cada capa del sistema debe validar sus entradas y proteger sus invariantes. El cliente valida antes de enviar; el servicio valida antes de procesar; las entidades validan su propio estado.

La degradación elegante permite que el sistema continúe operando de manera limitada cuando algunos componentes fallan. Si la persistencia no está disponible, el sistema puede continuar en memoria con advertencias apropiadas.

La recuperabilidad asegura que errores temporales pueden resolverse mediante reintentos y que errores permanentes se reportan claramente para intervención del usuario.

12.2 Validación de Operaciones

Validación de Construcción

Antes de construir una instalación, el sistema verifica que el jugador tiene Solaris suficientes para el coste de construcción, que el enclave destino tiene hectáreas disponibles para la ocupación de la instalación, que el código de instalación especificado es válido y existe, que el tipo de instalación es compatible con el tipo de enclave, y que la instalación no violaría ninguna otra restricción.

Los mensajes de error deben ser específicos y accionables. En lugar de "operación fallida", el sistema debe indicar "fondos insuficientes: requiere 1000 Solaris, disponible 750".

Validación de Traslado

Antes de trasladar una criatura, el sistema verifica que la criatura existe y está en una instalación válida, que la criatura ha alcanzado la edad adulta, que la criatura tiene salud igual o mayor a setenta y cinco, que la instalación destino existe y tiene espacio disponible, que la criatura es compatible con el medio de la instalación destino, que la criatura es compatible con el tipo de alimentación de la instalación destino, y que el jugador tiene Solaris suficientes para el coste de traslado.

Validación de Persistencia

Al cargar una partida guardada, el sistema debe verificar que los datos son completos, que las referencias entre entidades son válidas, que los valores numéricos están en rangos esperados, y que la versión del esquema de datos es compatible con el código actual.

12.3 Manejo de Errores de Comunicación

En sistemas distribuidos reales, las comunicaciones pueden fallar por razones de red, timeouts, o indisponibilidad de servicios. El código debe manejar estos casos gracefully.

Para errores transitorios como timeouts de red, el sistema puede implementar políticas de reintento con backoff exponencial. Un primer intento fallido se reintenta después de un segundo; si vuelve a fallar, después de dos segundos; y así sucesivamente hasta un máximo de intentos o tiempo total.

Para errores permanentes como servicio no disponible, el sistema debe reportar el error claramente y no reintentar automáticamente, ya que esto no resolvería el problema subyacente.

Para errores de serialización, el sistema debe registrar el mensaje problemático para debugging y reportar un error al usuario indicando que los datos pueden estar corruptos.

12.4 Registro y Trazabilidad

Todo error debe registrarse con suficiente contexto para diagnóstico posterior. El registro debe incluir timestamp preciso, tipo de error, mensaje descriptivo, stack trace completo si aplica, identidad del usuario o sesión si aplica, y estado del sistema en el momento del error.

Los logs estructurados en formato JSON facilitan el análisis automatizado posterior. Cada entrada de log incluye campos definidos que permiten filtrado y agregación eficiente.

La trazabilidad permite seguir el flujo de una operación a través de múltiples componentes. Un identificador de correlación presente en todas las entradas de log relacionadas facilita esta correlación.

12.5 Estados de Error

El diseño debe considerar estados de error como parte normal del espacio de estados, no como anomalías a evitar. Algunas situaciones genuinamente requieren que el sistema reporte error: fondos insuficientes, recursos agotados, operaciones inválidas.

Los resultados de operaciones deben ser tipos discriminated que distingan entre éxito con datos, fallo con razón específica, y situaciones excepcionales. El código cliente puede entonces manejar cada caso apropiadamente.

Los errores recoverable deben permitir reintento después de corrección. Los errores permanenterequieren intervención del usuario. El sistema debe hacer esta distinción clara.

13. ESTRUCTURAS DE DATOS Y ALGORITMOS

13.1 Selección de Estructuras

La selección de estructuras de datos impacta directamente en el rendimiento y mantenibilidad del código. Cada estructura se elige considerando los patrones de acceso predominantes en el sistema.

Las listas ordenadas son apropiadas para colecciones pequeñas donde el orden de iteración importa o donde se realizan frecuentemente inserciones al final. La complejidad de acceso por índice es O, pero la búsqueda es O(n).

Los diccionarios son apropiados para búsquedas frecuentes por clave. La complejidad de acceso, inserción y eliminación es O(1) en promedio. El costo es mayor consumo de memoria y orden de iteración no garantizado.

Los conjuntos son apropiados para verificar pertenencia cuando no se necesitan valores asociados. La complejidad de búsqueda es O(1) en promedio.

Los árboles balanceados proporcionan orden garantizado y complejidad O(log n) para operaciones básicas. Son apropiados cuando se necesita iteración ordenada junto con búsquedas frecuentes.

13.2 Estructuras para Entidades del Juego

Gestión de Criaturas

Las criaturas se almacenan en múltiples estructuras según el propósito de acceso. Un diccionario principal por identificador permite acceso directo O(1) desde cualquier contexto que tenga el ID. Listas por instalaciónagrupan criaturas según su ubicación

actual, facilitando el procesamiento de alimentación. Listas ordenadas por salud permiten identificar rápidamente criaturas que requieren atención.

La normalización del almacenamiento evita duplicación de datos y mantiene consistencia. La criatura existe una vez; referencias en instalaciones, enclaves y resultados de consulta apuntan a esa instancia.

Gestión de Instalaciones

Las instalaciones se agrupan primariamente por enclave. Cada enclave mantiene una lista de sus instalaciones, permitiendo iteración sobre todas las instalaciones de un enclave específico.

Un diccionario global de instalaciones por identificador permite acceso directo desde cualquier contexto. Este diccionario facilita la validación de referencias sin necesidad de buscar a través de enclaves.

Gestión de Eventos

Los eventos se almacenan en orden cronológico, típicamente mediante una lista donde nuevas entradas se añaden al final. Esta estructura optimiza la operación más frecuente, que es añadir eventos, mientras mantiene el orden natural para presentación.

Para consultas por rango de tiempo, un índice secundario por timestamp permite búsqueda eficiente sin escanear toda la lista.

13.3 Algoritmos Principales

Procesamiento de Alimentación

El algoritmo de procesamiento de alimentación itera sobre todas las instalaciones y sus criaturas, calculando ingesta requerida según fórmulas basadas en edad y especie. Para cada criatura, determina la disponibilidad de suministros en la instalación, calcula el porcentaje de requerimiento satisfecho, y aplica el efecto correspondiente a la salud.

La complejidad es $O(I \times C)$ donde I es el número de instalaciones y C es el número promedio de criaturas por instalación. Para partidas típicas con cientos de criaturas, este procesamiento es rápido incluso en implementaciones simples.

La optimización potencial incluye precalcular requisitos totales por instalación antes de distribuir suministros, evitando recalcular proporciones para cada criatura individualmente.

Procesamiento de Visitantes

El algoritmo de procesamiento de visitantes calcula llegadas y abandonos según fórmulas documentadas, actualizando la población de visitantes del enclave. Luego itera sobre instalaciones de exhibición procesando elecciones de favoritos.

Para cada criatura en cada instalación de exhibición, el algoritmo genera un número aleatorio para determinar si algún visitante la elige como favorita. Si es así, calcula y otorga la donation correspondiente.

La generación de números aleatorios debe ser determinista dada una semilla, permitiendo reproducibilidad de simulaciones para debugging o replays.

Serialización JSON

La serialización a JSON convierte objetos .NET en representaciones de texto que pueden guardarse en archivos o transmitirse. El proceso requiere considerar referencias circulares, tipospolimórficos, y propiedades no serializables.

El modelo de entidades debe diseñarse para serialización directa o implementar serializadores personalizados donde el estándar no sea suficiente.

La deserialización debe validar los datos cargados, detectando estados corruptos o incompatibles antes de que causen errores en tiempo de ejecución.

13.4 Consideraciones de Rendamiento

Persistencia Diferida

Guardar el estado completo después de cada operación individual genera overhead de I/O excesivo. El sistema puede diferir persistencia, guardando solo después de operaciones significativas o cada cierto número de operaciones.

Un flag de "dirty" marca cuando el estado ha cambiado desde el último guardado. Solo los estados dirty requieren escritura. El sistema puede also implementar auto-save periódico como respaldo adicional.

Procesamiento por Lotes

En lugar de procesar criaturas una a una, el procesamiento por lotes acumula cambios y los aplica al final. Esto reduce overhead de gestión de estado pero requiere cuidado para mantener consistencia si el procesamiento se interrumpe.

Concurrencia

Si el sistema evoluciona a arquitectura con múltiples procesos, el acceso concurrente a datos compartidos requiere sincronización. Candados mutex, operaciones atómicas, o estructuras de datos sin locks各有 trade-offs de rendimiento y complejidad.

---

14. PLAN DE DESARROLLO

14.1 Estructura de Entregas

El desarrollo se organiza en cuatro entregas incrementales que demuestran progreso y permiten retroalimentación temprana.

La primera entrega, Diseño de Dominio y Arquitectura, tiene peso del diez por ciento sobre la nota final con fecha recomendada del diecisiete de abril de dos mil veintiséis. El enfoque es análisis y diseño más que código funcional.

La segunda entrega, Comunicación, Persistencia y Prototipo Funcional Mínimo, tiene peso del veinte por ciento con fecha del ocho de mayo. El enfoque es infraestructura de comunicación y persistencia con funcionalidad básica.

La tercera entrega, Simulación, Concurrencia y Consistencia, tiene peso del veinte por ciento con fecha del veintidós de mayo. El enfoque es la lógica de juego completa y manejo de casos edge.

La entrega final, Sistema Completo y Memoria Técnica, tiene peso del cincuenta por ciento con fecha límite del tres de junio. El enfoque es integración total, documentación, y polish.

14.2 Entrega 1: Análisis y Diseño

Objetivos

El estudiante debe presentar un modelo de dominio completo que capture todas las entidades del juego, sus atributos, y relaciones. Este modelo debe evolucionar durante la implementación pero comienza aquí.

La estructura inicial de la solución debe definir los proyectos .NET que existirán, sus dependencias, y cómo se organizan los namespaces.

Las definiciones de entidades, enumerados, DTOs y contratos de interfaz deben estar suficientemente detallados para guiar la implementación posterior.

La justificación de arquitectura distribuida debe explicar qué componentes existirán, cómo se comunicarán, y qué responsabilidades asumirá cada uno.

Entregables

El documento de modelo de dominio describe las entidades con sus atributos, tipos de datos, y relaciones. Incluye justificación de decisiones de diseño.

El diagrama de clases muestra la estructura estática del sistema con clases, herencia, y asociaciones.

El documento de arquitectura justifica la separación en componentes propuesta, discutiendo alternativas consideradas y razones de la elección.

La especificación de contratos de interfaz define cada método disponible entre componentes con parámetros, tipos de retorno, y precondiciones.

14.3 Entrega 2: Infraestructura Básica

Objetivos

Un sistema mínimamente ejecutable permite crear una nueva partida, storing su estado, y recoverlo. Este es el esqueleto sobre el cual se construye la lógica de juego.

La primera implementación del cliente de administración proporciona una interfaz usable para las operaciones básicas. Puede ser simple pero debe funcionar correctamente.

El mecanismo de comunicación entre componentes demuestra que los límites de proyecto se respetan y los componentes pueden intercambiar datos.

Entregables

Código fuente compilable que demuestra las capacidades básicas. El código debe ser limpio, documentado donde sea necesario, y organizado según la estructura diseñada.

Un manual de usuario básico explica cómo usar el sistema para crear partida, realizar operaciones simples, y guardar/cargar estado.

14.4 Entrega 3: Lógica Completa

Objetivos

La lógica principal de simulación implementa todas las fórmulas documentadas: alimentación de criaturas, cambios de salud, nacimientos, visitantes, donations.

El procesamiento de rondas completo sigue la secuencia de fases definida, generando resultados coherentes.

La gestión de criaturas incluye reproducción, envejecimiento, y todas las validaciones necesarias.

Los visitantes generan прибыль para el jugador según el diseño.

El registro de eventos captura todas las acciones significativas con detalle suficiente.

Entregables

Código fuente completo con toda la lógica de juego. El código debe demostrar manejo de casos edge y tratamiento de errores.

Un registro de eventos de muestra que demuestra el funcionamiento del sistema con datos de prueba.

Un informe de testing que documenta las pruebas realizadas y los resultados obtenidos.

14.5 Entrega Final: Producto Completo

Objetivos

La solución completa debe estar fully compilable sin errores ni warnings significativos. El código debe estar limpio, documentado, y organizado.

La memoria técnica documenta todas las decisiones de diseño, implementación, y testing realizadas durante el desarrollo.

La demostración del sistema muestra todas las funcionalidades trabajando correctamente.

Apartados de la Memoria

El análisis de la aplicación presenta la organización general de la solución y los objetivos funcionales del sistema.

Los fundamentos distribuidos aplicados explican la relación entre la teoría del curso y las decisiones adoptadas en la práctica.

El diseño de arquitectura detalla los proyectos, servicios, contratos, y comunicación entre componentes.

La sección de concurrencia y sincronización describe las estrategias utilizadas para coordinar procesos, tareas o recursos compartidos.

La sección de persistencia y consistencia explica la estrategia de guardado, recuperación, serialización, y coherencia entre componentes.

La sección de seguridad detalla las medidas básicas de validación, protección de datos, y control de errores.

Las estructuras de datos justifican los tipos utilizados y las razones de su elección.

Las técnicas de ordenación y búsqueda implementedse documentan con motivo de elección.

Los diagramas UML o equivalentes proporcionan una representación visual de la estructura del sistema.

Los fallos conocidos documentan funcionalidades no implementadas o limitaciones del sistema actual.

Las conclusiones proporcionan una valoración general del trabajo realizado y el tiempo dedicado.

---

15. APÉNDICES

15.1 Catálogo Completo de Especies

Especie	Medio	Alimentación	Apetito Base	Edad Adulta	σ Traslado
Gusano Arena	SUBTERRÁNEO	DEPREDADOR	5	24	25
Tigre Laza	DESIERTO	DEPREDADOR	8	38	5
Muad'Dib	DESIERTO	RECOLECTOR	2	12	5
Halcón Desierto	AÉREO	DEPREDADOR	2	16	15
Trucha Arena	SUBTERRÁNEO	RECOLECTOR	10	42	25

15.2 Catálogo Completo de Instalaciones

Código	Tipo	Coste	Hectáreas	Capacidad	Suministros Iniciales
ADR05	ACLIMATACIÓN	1,000	10	5	200
ADP03	ACLIMATACIÓN	2,500	50	3	300
AAV02	ACLIMATACIÓN	5,000	100	2	500
ASU04	ACLIMATACIÓN	3,500	25	4	100
EDR02	EXHIBICIÓN	21,000	200	2	0
EDP03	EXHIBICIÓN	1,250	300	3	0
EAV02	EXHIBICIÓN	15,000	200	2	0

Código	Tipo	Coste	Hectáreas	Capacidad	Suministros Iniciales
ESU03	EXHIBICIÓN	25,000	400	3	0

15.3 Configuración de Escenarios

Escenario	Solaris	Hectáreas	Visitantes	Nivel Adquisitivo
Arrakeen	100,000	500	2,000	ALTO
Giedi Prime	50,000	100	2,000	BAJO
Caladan	150,000	10,000	3,000	MEDIO

15.4 Glosario de Términos

α (alpha): Multiplicador de ingesta según tipo de instalación. Vale 1 en exhibición, 15 en aclimatación.

σ (sigma): Multiplicador de donation según nivel adquisitivo del enclave. Vale 1 (BAJO), 15 (MEDIO), 30 (ALTO).

CHOAM: Corps Handlers of Arrakis and Mankind. Corporación dominante del universo Dune.

Letargo: Estado irreversible de criatura cuya salud llegó a cero.

Melange: Especia, recurso más valioso del universo Dune.

Ronda: Ciclo mensual de simulación en el juego.

Solaris: Moneda estándar del universo Dune.

Nota Final

Este documento define el diseño completo del proyecto Dune: Arrakis Dominion Distributed. Las especificaciones aquí contenidas deben guiar la implementación mientras permiten flexibilidad para mejoras durante el desarrollo.

El éxito del proyecto depende no solo de implementar las mecánicas descritas, sino de capturar el espíritu del universo Dune en una experiencia de gestión satisfactoria y memorable.

Documento preparado para CHOAM Distributed Entertainment
Basado en el universo creado por Frank Herbert
Práctica académica de Programación en Entornos Distribuidos - 2026